DSA — MODULE 5

Separate colourful exam notes • Graphs and Graph Algorithms

Prepared from the graph-related lecture files you uploaded: Graph Introductions, Graph Traversals and Applications, Shortest Paths, Floyd's Algorithm, Minimum Spanning Trees, Topological Sort, and Biconnected Components.

> Coverage: graph definitions and representations • BFS/DFS • connected components • topological sorting • shortest paths • Floyd–Warshall • minimum spanning trees (Prim/Kruskal) • bridges, articulation points and biconnected components.

How to answer graph-algorithm questions

For each algorithm, write: purpose → key idea/data structure → numbered steps or pseudocode → trace on the given graph → time complexity → limitation/condition. In a trace, show the queue/stack or distance table after each important step.

1. Graph foundations

A graph is G = (V, E), where V is the set of vertices (nodes) and E is the set of edges connecting pairs of vertices. Your graph-introduction slides define graphs this way and distinguish adjacency-matrix and adjacency-list representations. ■filecite■turn13file2■L37-L42■

Term	Meaning
Vertex / node	An entity represented in the graph.
Edge	A connection between vertices.
Directed graph (digraph)	Edges have directions; (u,v) need not imply (v,u).
Undirected graph	Edges have no direction; connection is symmetric.
Weighted graph	Edges carry costs/weights.
Degree	Number of incident edges; in-degree counts incoming directed edges, out-degree counts outgoing edges.
Path	A sequence of vertices joined by edges.
Cycle	A path that returns to its starting vertex.
Connected graph	Every pair of vertices is connected by a path (for undirected graphs).
DAG	Directed acyclic graph; a directed graph with no directed cycle.

Graph representations

Representation	Storage	Advantages / disadvantages
Adjacency matrix	n × n matrix; A[i][j] indicates whether edge (i,j) exists.	Simple; edge lookup O(1); space $O(n^2)$, even for sparse graphs.
Adjacency list	Each vertex stores its neighbours.	Space O(V+E); good for sparse graphs; checking one edge can require scanning neighbours.

The supplied graph slides explicitly note the adjacency matrix's $O(n^2)$ space and the adjacency list's space efficiency proportional to vertices and edges. ■filecite■turn13file2■L220-L228■ ■filecite■turn13file2■L235-L281■

Trees and forests

The traversal slides define a free tree as a connected undirected graph without cycles, with |E| = |V| – 1. A forest is an acyclic graph consisting of trees. ■filecite■turn13file3■L54-L70■

2. Breadth-First Search (BFS)

BFS visits vertices level by level: first the start vertex, then its unvisited neighbours, then the neighbours of those vertices. The uploaded traversal slides implement BFS using a queue and a boolean visited/enqueued array. ■filecite■turn13file3■L131-L153■

Algorithm

```
BFS(G, s):
    mark s visited
    enqueue(s)
    while queue is not empty:
        v = dequeue()
        visit(v)
        for each neighbour u of v:
            if u is not visited:
                mark u visited
                enqueue(u)
```

Worked mini-example

Suppose the edges are A–B, A–C, B–D, B–E, C–F. Starting at A and visiting neighbours alphabetically:

Step	Queue after processing	Visited order
Start	[A]	—
Process A	[B, C]	A
Process B	[C, D, E]	A, B
Process C	[D, E, F]	A, B, C
Process D	[E, F]	A, B, C, D
Process E	[F]	A, B, C, D, E
Process F	[]	A, B, C, D, E, F

Complexity

Representation	Time
Adjacency list	O(V + E)
Adjacency matrix	$O(V^2)$

The uploaded BFS analysis gives O(m+n) for adjacency lists and $O(n^2)$ for adjacency matrices, where n=|V| and m=|E|. ■filecite■turn13file3■L215-L223■

> Applications: shortest paths in an unweighted graph, level-order exploration, reachability, and connected-component discovery.

3. Depth-First Search (DFS)

DFS follows one path as far as possible, then backtracks to explore remaining unvisited neighbours. It can be implemented recursively using the call stack or iteratively using an explicit stack. ■filecite■turn13file3■L232-L248■

Recursive preorder DFS

```
DFS(v):
    mark v visited
    visit(v)
    for each neighbour u of v:
        if u is not visited:
            DFS(u)
```

Preorder vs postorder

Traversal	When vertex is processed	Typical use
Preorder DFS	When first discovered	Build DFS tree; discover reachable nodes.
Postorder DFS	After all unvisited descendants are processed	Finish ordering; useful in graph analysis.

Disconnected graphs

A traversal started from one vertex may not reach every vertex. To visit all vertices, loop through every vertex and start a traversal whenever that vertex has not been visited; the supplied traversal slides call this the all-vertices approach. ■filecite■turn13file3■L93-L110■

BFS vs DFS	BFS	DFS
Core structure	Queue (FIFO)	Stack or recursion
Exploration	Level by level	Deep path, then backtrack
Unweighted shortest path	Finds minimum-edge paths	Does not generally guarantee shortest path
Adjacency-list complexity	O(V+E)	O(V+E)

4. Topological Sort

Topological sorting arranges vertices of a directed acyclic graph (DAG) so every vertex appears before its successors. It models prerequisite/dependency order. The uploaded slides explicitly define it for DAGs and note that the result need not be unique. ■filecite■turn13file6■L45-L62■

Kahn's algorithm (in-degree method)

1. Compute the in-degree of every vertex.

2. Put all vertices with in-degree 0 into a queue (or priority queue).

3. Remove one such vertex, append it to the output order, and reduce the in-degree of each outgoing neighbour.

4. Whenever a neighbour's in-degree becomes 0, add it to the queue.

5. If all vertices are output, the order is valid. If fewer than |V| vertices are output, the graph contains a cycle.

```
TopologicalSort(G):
    compute indegree[v] for every vertex v
    queue = all vertices with indegree 0
    order = []
    while queue not empty:
        v = remove one vertex from queue
        append v to order
        for each edge v -> u:
            indegree[u] -= 1
            if indegree[u] == 0:
                add u to queue
    if len(order) != |V|: report cycle
    else return order
```

Important facts

Question	Answer
Can a graph with a directed cycle have a topological ordering?	No.
Is the ordering always unique?	No; multiple zero-in-degree choices may be possible.
Time complexity with adjacency lists	O(V+E).
When is it useful?	Task scheduling, course prerequisites, build/dependency order.

The uploaded implementation uses an in-degree array and a queue/priority queue; it returns the number of vertices visited so a cycle can be detected if not all vertices are processed. ■filecite■turn13file6■L166-L212■

5. Shortest Paths — Dijkstra's Algorithm

A shortest-path problem seeks a path whose total edge weight is minimum. The supplied shortest-path lecture defines path length as the sum of the weights of its edges. ■filecite■turn13file5■L9-L17■

Dijkstra: key idea

Maintain a tentative distance from the source to each vertex. Repeatedly choose the unvisited vertex with the smallest tentative distance, finalize it, and relax its outgoing edges. Dijkstra's algorithm requires non-negative edge weights.

Relaxation rule

```
if dist[u] + weight(u, v) < dist[v]:
    dist[v] = dist[u] + weight(u, v)
    parent[v] = u
```

Algorithm steps

1. Set dist[source] = 0; set every other distance to ∞.

2. Select the unvisited vertex u with the smallest tentative distance.

3. Mark u as processed.

4. For each edge u → v, update dist[v] if dist[u] + w(u,v) is smaller.

5. Repeat until all reachable vertices are processed or the destination is finalized.

Mini-trace

Vertex	Current distance	Reason / predecessor
S	0	Source
A	4	Direct edge S→A of weight 4
B	2	Direct edge S→B of weight 2
C	5	Best route S→B→C with weights 2+3

> Warning: Dijkstra is not correct for graphs with negative edge weights. For negative edges without negative cycles, use a suitable algorithm such as Bellman–Ford; this is general algorithm context beyond the uploaded lecture's stated non-negative-weight focus.

6. Floyd–Warshall Algorithm

Floyd's algorithm solves the all-pairs shortest-path problem: it finds shortest distances between every ordered pair of vertices. The uploaded slides allow negative edge weights but assume there are no negative cycles. ■filecite■turn13file1■L38-L48■

Initialization

Create a weight matrix W: W[i][i]=0; W[i][j]=weight(i,j) if the edge exists; otherwise W[i][j]=∞. Begin with D■■■ = W.

Dynamic programming recurrence

```
D^(k)[i,j] = min(
    D^(k-1)[i,j],
    D^(k-1)[i,k] + D^(k-1)[k,j]
)
```

Interpretation: when vertex k is newly allowed as an intermediate, either the best path avoids k, or it goes from i to k and then k to j. This is the recurrence shown in the uploaded Floyd lecture. ■filecite■turn13file1■L141-L174■

In-place pseudocode

```
D = W
for k = 0 to n-1:
    for i = 0 to n-1:
        for j = 0 to n-1:
            D[i][j] = min(D[i][j], D[i][k] + D[k][j])
```

Feature	Floyd–Warshall
Problem	All-pairs shortest paths
Time	$O(V^3)$
Space	$O(V^2)$ for distance matrix
Negative edges	Allowed if no negative cycle
Negative-cycle check	If any D[i][i] becomes negative after processing, a negative cycle exists.

The source also introduces a pointer array P that records the intermediate vertex responsible for an improvement, allowing the actual shortest path to be reconstructed. ■filecite■turn13file1■L200-L207■

7. Minimum Spanning Trees (MST)

For a connected, weighted, undirected graph, a spanning tree includes every vertex, has no cycle, and contains |V|–1 edges. A minimum spanning tree is a spanning tree with minimum total edge weight. These are the properties stated in the uploaded MST lecture. ■filecite■turn13file4■L53-L72■

Prim's algorithm — grow one tree

1. Start with any vertex.
2. Repeatedly choose the minimum-weight edge connecting a vertex already in the tree to a vertex outside it.
3. Add that edge and the new vertex.
4. Stop after all vertices are included (|V|–1 edges).

Kruskal's algorithm — grow a forest

1. Sort all edges in nondecreasing order of weight.
2. Start with each vertex in its own component.
3. Consider edges from smallest to largest; accept an edge only if its endpoints belong to different components.
4. Use Union-Find (Disjoint Set) to detect whether endpoints are already connected.
5. Stop when |V|–1 edges have been accepted.

The uploaded MST slides describe Prim as incrementally expanding a tree and Kruskal as building a forest while rejecting edges that create cycles. ■filecite■turn13file4■L108-L112■ ■filecite■turn13file4■L200-L220■

Prim vs Kruskal	Prim	Kruskal
Starting point	Choose one starting vertex	Start with a forest of single-vertex trees
Choice	Cheapest edge leaving the current tree	Cheapest remaining edge that joins different components
Cycle handling	Avoid adding a cycle by adding a new vertex	Explicitly reject edges whose endpoints are already connected
Common implementation	Priority queue / min-heap	Sort or priority queue + Union-Find
Typical complexity	O((V+E) log V) with suitable heap/adjacency-list implementation	O(E log E), dominated by edge sorting (Union-Find near-constant amortized)

Worked Kruskal example

Edges: AB=1, BC=2, AC=3, CD=4. Sort by weight: AB, BC, AC, CD. Accept AB and BC; reject AC because A and C are already connected (it would create a cycle); accept CD. MST edges = {AB, BC, CD}, total cost = 1+2+4 = 7.

> MST is not the same as a shortest-path tree: an MST minimizes the total weight connecting all vertices, while a shortest-path tree minimizes path distance from a chosen source.

8. Biconnected Components, Bridges and Separating Vertices

The uploaded biconnected-components slides describe a biconnected component in terms of a maximal edge set linked by simple cycles, or a single edge; a bridge is treated there as a component of size one, and a separating vertex is one whose deletion partitions the graph. ■filecite■turn13file0■L69-L78■

DFS numbering: pre and low

pre[u] is the DFS discovery/preorder number of u. **low[u]** records the smallest discovery number reachable from u's DFS subtree using tree edges downward and at most one back edge. The slide's low-numbering formula considers pre(u) and discovery numbers reached by back edges from descendants. ■filecite■turn13file0■L158-L176■

Articulation / separating vertex test

For a DFS tree, a non-root vertex u is an articulation point if it has a child v with low[v] ≥ pre[u]. The DFS root is an articulation point if it has more than one DFS-tree child. This is the standard test associated with the low/pre method introduced in the uploaded slides.

Bridge test

For a DFS tree edge (u,v), where u is the parent of v, the edge is a bridge if low[v] > pre[u]: the subtree rooted at v has no back edge that reaches u or an ancestor of u.

High-level algorithm

1. Run DFS and assign pre[] discovery numbers.
2. Compute low[] values using child low values and back edges.
3. Use low/pre comparisons to identify separating vertices and bridges.
4. Use a second DFS and an edge stack to output biconnected components as edge sets; the lecture emphasizes that components are edges, not vertices.

The source explicitly states that DFS can find pre/low values and separating vertices, then a second DFS can identify biconnected components; it stresses that components are sets of edges. ■filecite■turn13file0■L129-L138■

> Careful: A bridge is an edge whose removal disconnects its connected component. An articulation point is a vertex whose removal increases the number of connected components. Do not confuse these two.

9. Algorithm comparison — one-page revision

Algorithm	Problem solved	Core idea / data structure	Complexity
BFS	Reachability; unweighted shortest path	Queue; level-by-level	O(V+E) with adjacency list
DFS	Reachability; graph structure analysis	Stack or recursion; backtracking	O(V+E) with adjacency list
Topological sort	Dependency order in a DAG	Repeatedly process in-degree-0 vertices	O(V+E)
Dijkstra	Single-source shortest paths, non-negative weights	Greedy minimum tentative distance + relaxation	O((V+E) log V) with heap
Floyd–Warshall	All-pairs shortest paths	Dynamic programming over intermediate vertices	$O(V^3)$ time, $O(V^2)$ space
Prim	Minimum spanning tree	Grow one connected tree	O((V+E) log V) with heap
Kruskal	Minimum spanning tree	Sorted edges + Union-Find	O(E log E)
Biconnected analysis	Bridges/articulation points/components	DFS discovery pre[] and low[]	O(V+E) with adjacency lists

10. Exam-style questions

1. Define a graph. Compare adjacency matrix and adjacency list with space/time trade-offs.
2. Explain BFS with pseudocode and trace the queue for a given graph.
3. Explain DFS preorder and postorder. How do you visit all vertices in a disconnected graph?
4. Define a DAG and explain topological sorting using the in-degree method.
5. Run Dijkstra's algorithm from a given source and show the distance/predecessor table after each iteration.
6. Explain Floyd–Warshall's recurrence and calculate the final all-pairs distance matrix.
7. Define a spanning tree and MST. Construct an MST using Prim's algorithm.
8. Apply Kruskal's algorithm; show accepted/rejected edges and the final cost.
9. Explain biconnected components, bridges and separating vertices using DFS.
10. What are pre[] and low[] values? State the articulation-point and bridge conditions.

11. Last-minute memory sheet

Concept	Remember
BFS	Queue; level by level; unweighted shortest paths.
DFS	Stack/recursion; deep first; backtrack; pre/post.
Topo sort	DAG only; in-degree zero; fewer than V outputs means cycle.
Dijkstra	Single source; non-negative weights; relax edges.
Floyd–Warshall	All pairs; min(D[i][j], D[i][k]+D[k][j]); $O(V^3)$.
MST	Connected weighted undirected graph; V–1 edges; no cycle.
Prim	Grow one tree with cheapest outgoing edge.
Kruskal	Sort edges; skip cycles; Union-Find.
Biconnected	DFS pre/low; articulation points, bridges; edge components.

Scope note: This separate Module 5 PDF is based on the graph-related files you uploaded. The duplicate Topological Sort/Topological-sort slide decks are treated as one topic.